

LOGMIC V1



Romain DUCHADEAU (IP2I / CNRS) ¹

Edouard BECHETOILLE (IP2I / CNRS) ²

Herve MATHEZ (IP2I / CNRS) ³

2025-2027

1. r.duchadeau@ip2i.in2p3.fr

2. e.bechetoille@ip2i.in2p3.fr

3. h.mathez@ip2i.in2p3.fr

Table des matières

1. Simulation RTL avec Xcelium — Du full adder au testbench réutilisable	3
1.1. Progression de la simulation	3
1.1.1. Les trois phases	3
1.1.2. Méthode en quatre niveaux	3
1.2. Full adder combinatoire	4
1.2.1. Contrat logique	4
1.2.2. RTL synthétisable	4
1.2.3. Testbench exhaustif	4
1.3. Filelists et commande Xcelium	5
1.3.1. Filelist RTL	5
1.3.2. Filelist testbench	5
1.3.3. Commande minimale	5
1.4. Wrapper réutilisable	6
1.4.1. Waveforms optionnelles	6
1.4.2. Wrapper contrôlé	6
1.5. Augmenter la complexité	7
1.5.1. Additionneur pipeline	7
1.6. Diagnostic	8
1.7. Checklist finale	8

1. Simulation RTL avec Xcelium — Du full adder au testbench réutilisable

test :

```
1 pub fn main() {
2     println!("Hello, world!");
3 }
```

rust

autre test :

```
1 def fib(n):
2     if n <= 1:
3         return n
4     else:
5         return f{fib(n - (a)1)} + f{fib(n - 2) (b)}
6 print(fib(25))
```

py

1.1. Progression de la simulation

1.1.1. Les trois phases

1. **Compilation** : lecture du SystemVerilog et contrôle de syntaxe.
2. **Élaboration** : construction de la hiérarchie et résolution des modules, paramètres et ports.
3. **Simulation** : exécution des blocs temporels et production du verdict du testbench.

Une erreur de syntaxe appartient généralement à la compilation. Un top inconnu ou un port impossible appartient à l'élaboration. Un \$fatal, un timeout ou une comparaison incorrecte appartient à la simulation.

1.1.2. Méthode en quatre niveaux

| Niveau | Objectif | | 1 | Écrire un petit DUT et un testbench auto-vérifiant. | | 2 | Séparer RTL et testbench avec deux filelists. | | 3 | Lancer une commande xrun directe et comprendre chaque option. | | 4 | Ajouter un wrapper, des logs, des waveforms et un second design séquentiel. |

—

1.2. Full adder combinatoire

1.2.1. Contrat logique

Pour les entrées a, b et c_in :

- $\text{sum} = a \text{ XOR } b \text{ XOR } c_{\text{in}}$
- $c_{\text{out}} = ab + ac_{\text{in}} + bc_{\text{in}}$

a	b	c_in	sum	c_out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

1.2.2. RTL synthétisable

```

1  `timescale 1ns/1ps
2
3  module full_adder_comb (
4      input  logic a_i,
5      input  logic b_i,
6      input  logic cin_i,
7      output logic sum_o,
8      output logic cout_o
9  );
10     assign sum_o = a_i ^ b_i ^ cin_i;
11     assign cout_o = (a_i & b_i)
12                     | (a_i & cin_i)
13                     | (b_i & cin_i);
14 endmodule

```

Le DUT ne contient aucun délai, bloc initial ou appel système. Ces éléments appartiennent au testbench.

1.2.3. Testbench exhaustif

```

1  `timescale 1ns/1ps
2
3  module tb_full_adder_comb;
4      logic a_i, b_i, cin_i;
5      logic sum_o, cout_o;
6
7      full_adder_comb dut (
8          .a_i(a_i), .b_i(b_i), .cin_i(cin_i),
9          .sum_o(sum_o), .cout_o(cout_o)
10     );
11
12     initial begin : test_exhaustif
13         logic sum_attendue;
14         logic cout_attendue;
15
16         for (int vecteur = 0; vecteur < 8; vecteur++) begin

```

```

17      {a_i, b_i, cin_i} = vecteur[2:0];
18      =10;
19
20      sum_attendue = a_i ^ b_i ^ cin_i;
21      cout_attendue = (a_i & b_i)
22                      | (a_i & cin_i)
23                      | (b_i & cin_i);
24
25      if ({cout_o, sum_o} !==
26          {cout_attendue, sum_attendue}) begin
27          $fatal(1,
28              "vecteur=%03b obtenu=%b%b attendu=%b%b",
29              {a_i,b_i,cin_i}, cout_o, sum_o,
30              cout_attendue, sum_attendue);
31      end
32  end
33
34  $display("TEST_PASS: full_adder_comb");
35  $finish;
36  end
37 endmodule

```

> **À retenir** : Un testbench automatisable doit terminer avec \$fatal en cas d'erreur et imprimer un marqueur final exact uniquement lorsque tous les tests passent.

—

1.3. Filelists et commande Xcelium

1.3.1. Filelist RTL

rtl.f

```
1 rtl/full_adder_comb.sv
```

text

1.3.2. Filelist testbench

tb.f

```
1 tb/tb_full_adder_comb.sv
```

text

Les chemins sont interprétés depuis le dossier où xrun est lancé. Une méthode simple consiste à lancer la commande depuis la racine du projet.

1.3.3. Commande minimale

```

1 xrun -64bit -sv -timescale 1ns/1ps \
2   -f sim/rtl.f \
3   -f sim/tb.f \

```

bash

```
4 -top tb_full_adder_comb \
5 -R
```

- -64bit utilise l'exécutable 64 bits.
- -sv active SystemVerilog.
- -timescale fixe l'unité et la précision par défaut.
- -f lit une filelist.
- -top choisit le top élaboré.
- -R lance la simulation après l'élaboration.

—

1.4. Wrapper réutilisable

1.4.1. Waveforms optionnelles

Pour une petite simulation, la visibilité complète peut être activée avec -access +rwc. Un fichier Tcl Xcelium peut créer une base SHM :

```
1 database -open waves -into waves.shm -default
2 probe -create tb_full_adder_comb -all -depth all
3 run
4 exit
```

tcl

Puis :

```
1 xrun ... -input waves.tcl
2 simvision waves.shm
```

bash

1.4.2. Wrapper contrôlé

Le wrapper suivant crée un dossier de résultat unique, isole la bibliothèque Xcelium, conserve le code retour et vérifie le marqueur final.

```
1 =!/usr/bin/env bash
2 set -Eeuo pipefail
3
4 XRUN_BIN=${XRUN_BIN:-xrun}
5 RUN_ROOT=${RUN_ROOT:-/tmp/digi_tuto_runs}
6 RUN_ID=${RUN_ID:-$(date -u +%Y%m%dT%H%M%S)}
7 RUN_DIR="$RUN_ROOT/xcelium/full_adder_comb/$RUN_ID"
8
9 command -v "$XRUN_BIN" >/dev/null 2>&1 || {
10     echo "ERREUR: xrun introuvable" >&2
11     exit 127
12 }
13
14 mkdir -p "$RUN_DIR"
```

bash

```

15
16 set +e
17 "$XRUN_BIN" \
18 -64bit -sv -timescale 1ns/1ps \
19 -access +rwc \
20 -f sim/rtl.f \
21 -f sim/tb.f \
22 -top tb_full_adder_comb \
23 -xmlbdirname "$RUN_DIR/xcelium.d" \
24 -logfile "$RUN_DIR/xrun.log" \
25 -R
26 XRUN_RC=$?
27 set -e
28
29 if (( XRUN_RC != 0 )); then
30     echo "RESULTAT: FAIL (xrun=$XRUN_RC)" >&2
31     exit "$XRUN_RC"
32 fi
33
34 grep -Fxq "TEST_PASS: full_adder_comb" \
35 "$RUN_DIR/xrun.log" || {
36     echo "RESULTAT: FAIL (marqueur absent)" >&2
37     exit 1
38 }
39
40 echo "RESULTAT: PASS"
41 echo "LOG=$RUN_DIR/xrun.log"

```

1.5. Augmenter la complexité

1.5.1. Additionneur pipeline

Un second exemple ajoute une clock, un reset synchrone, des registres et un protocole valid. Une transaction acceptée avec valid_i=1 produit un résultat valide un cycle plus tard.

```

1  module adder_pipeline =(
2      parameter int unsigned WIDTH = 8
3  ) (
4      input logic          clk_i,
5      input logic          rst_ni,
6      input logic          valid_i,
7      input logic [WIDTH-1:0] a_i,
8      input logic [WIDTH-1:0] b_i,
9      input logic          cin_i,
10     output logic          valid_o,

```

```

11  output logic [WIDTH-1:0] sum_o,
12  output logic          cout_o
13 );
14  logic [WIDTH:0] result_q;
15  logic          valid_q;
16
17  always @(posedge clk_i) begin
18      if (!rst_ni) begin
19          result_q <= '0;
20          valid_q  <= 1'b0;
21          sum_o    <= '0;
22          cout_o   <= 1'b0;
23          valid_o  <= 1'b0;
24      end else begin
25          result_q <= {1'b0,a_i}
26                  + {1'b0,b_i}
27                  + cin_i;
28          valid_q  <= valid_i;
29          {cout_o,sum_o} <= result_q;
30          valid_o  <= valid_q;
31      end
32  end
33 endmodule

```

Le scoreboard doit décaler la référence d'un cycle. Les stimuli sont de préférence appliqués au front descendant afin d'éviter une course avec la capture au front montant.

—

1.6. Diagnostic

| Symptôme | Cause probable | Première action | |—|—|—| | xrun introuvable | environnement non chargé |
Vérifier command -v xrun. | | Fichier absent | chemin relatif incorrect | Ouvrir les filelists depuis le dossier de lancement. | | Top inconnu | mauvais argument -top | Vérifier le nom exact du module testbench. | | Port inconnu | interface DUT/TB différente | Comparer noms, directions et largeurs. | | Sortie X | signal non piloté ou reset absent | Chercher le premier temps où la valeur devient inconnue. | | Code retour nul sans PASS | testbench terminé trop tôt | Exiger le marqueur terminal exact. |

> **Ordre de debug** : Lire d'abord la première erreur de compilation, puis l'élaboration, puis le premier vecteur fonctionnel en échec. Ouvrir les waveforms seulement après avoir localisé la classe d'erreur.

—

1.7. Checklist finale

1. Le DUT est séparé du testbench.
2. Les deux filelists sont lisibles depuis le dossier de lancement.
3. Le top testbench est explicite.

-
4. Les huit vecteurs du full adder sont vérifiés.
 5. Le code retour Xcelium et le marqueur final sont tous les deux contrôlés.
 6. Les résultats sont écrits hors des sources.
 7. La GUI sert au diagnostic, pas au verdict.